Dokumentation FogCast App

Dokumentation FogCast App 1
1. Projektdokumentation 1
1.1. Einleitung 1
1.2. Anforderungsanalyse 2
1.2.1. Funktionale Anforderungen 2
1.2.2. Nicht-funktionale Anforderungen 3
1.2.3. Zielgruppe 4
1.3. Systemarchitektur 4
1.4. Technologie-Stack 7
1.5. Fazit und Ausblick 8
2. Technische Dokumentation 9
2.1. Softwarearchitektur 9
3.2 Einstiegspunkt der Anwendung 9
3.3 State Management 9
3.4 API-Schicht 10
3.5 Repository-Schicht 10
3.6 Datenmodelle (DTOs) 11
3.7 Benutzeroberfläche 11
3.8 Persistente Speicherung 11
3.9 Erweiterbarkeit 12

1. Projektdokumentation

1.1. Einleitung

Im Rahmen eines über zwei Semester angelegten Softwareprojekts wurde die mobile Anwendung FogCast entwickelt. Das Projekt baut auf den Ergebnissen einer vorherigen Projektgruppe auf, die im ersten Projektabschnitt eine Wetterstation konzipiert, aufgebaut und in Betrieb genommen sowie eine zugehörige Webanwendung zur Visualisierung der erfassten Wetterdaten entwickelt hat. Die Wetterstation misst verschiedene meteorologische Messgrößen wie Temperatur, Luftfeuchtigkeit, Luftdruck, Niederschlag und Windgeschwindigkeit und stellt diese über eine Programmierschnittstelle (API) zur Verfügung.

Auf Basis dieser bestehenden Infrastruktur bestand die Aufgabe der aktuellen Projektgruppe darin, eine mobile Anwendung zu entwickeln, die den Zugriff auf die erfassten Wetter- und Prognosedaten auch auf Smartphones ermöglicht. Ziel war es, eine benutzerfreundliche und moderne App bereitzustellen, die sowohl aktuelle Messwerte als auch Wettervorhersagen übersichtlich darstellt und dabei die Möglichkeiten mobiler Endgeräte optimal nutzt.

Für die Umsetzung wurde das Framework Flutter verwendet, das die Entwicklung plattformübergreifender Anwendungen für Android und iOS ermöglicht. Die App greift auf die bereits vorhandene Backend-Infrastruktur und die von der Wetterstation bereitgestellten Daten zu. Zusätzlich wurden Funktionen entwickelt, die über die bestehende Webanwendung hinausgehen. Dazu zählen unter anderem ein personalisierbarer Expertenmodus, verschiedene Diagrammtypen zur Visualisierung meteorologischer Daten sowie die Möglichkeit, individuelle Benutzereinstellungen dauerhaft zu speichern.

Das Projekt verfolgte sowohl technische als auch gestalterische Ziele. Neben der Anbindung an die bestehende Dateninfrastruktur standen die Entwicklung einer intuitiven Benutzeroberfläche, die Visualisierung zeitabhängiger Wetterdaten sowie die Umsetzung einer skalierbaren Softwarearchitektur im Vordergrund. Die vorliegende Dokumentation beschreibt die Anforderungen an die Anwendung, die zugrunde liegende Systemarchitektur, die eingesetzten Technologien sowie die Implementierung der wichtigsten Funktionen. Darüber hinaus werden Herausforderungen während der Entwicklung und mögliche zukünftige Erweiterungen der Anwendung vorgestellt.

1.2. Anforderungsanalyse

Die Entwicklung einer Softwareanwendung beginnt mit der Erhebung und Definition der Anforderungen, die durch das System erfüllt werden sollen. Die Anforderungsanalyse dient dazu, die Erwartungen der zukünftigen Nutzerinnen und Nutzer sowie die technischen Rahmenbedingungen systematisch zu erfassen und in konkrete Anforderungen zu überführen. Sie stellt damit die Grundlage für die spätere Konzeption, Implementierung und Evaluation der Anwendung dar.

1.2.1. Funktionale Anforderungen

Funktionale Anforderungen beschreiben die konkreten Funktionen und Dienste, die ein Softwaresystem bereitstellen muss. Für die Entwicklung der FogCast-App wurden die folgenden funktionalen Anforderungen definiert:

Anzeige aktueller Wetterdaten

Die Anwendung soll aktuelle Messwerte der Wetterstation abrufen und übersichtlich darstellen. Dazu gehören insbesondere Temperatur, Luftfeuchtigkeit, Luftdruck, Windgeschwindigkeit und Windrichtung.

Anzeige von Wettervorhersagen

Neben den aktuellen Messwerten soll die Anwendung Wettervorhersagen für mehrere Tage bereitstellen. Nutzer:innen sollen sowohl eine Tagesübersicht als auch detaillierte stündliche Prognosen einsehen können.

Detailansicht einzelner Tage

Für jeden Vorhersagetag soll eine separate Detailansicht verfügbar sein, in der die Wetterentwicklung im Tagesverlauf dargestellt wird. Hierzu werden stündliche Prognosedaten visualisiert.

Expertenmodus

Die Anwendung soll einen erweiterten Expertenmodus bereitstellen. Dieser ermöglicht die Darstellung zusätzlicher meteorologischer Parameter sowie die Analyse historischer Messdaten.

Individuelle Diagrammauswahl

Nutzer:innen sollen selbst festlegen können, welche Diagramme im Expertenmodus angezeigt werden. Dazu gehören unter anderem Temperatur, Luftfeuchtigkeit, Niederschlag, Wolkendichte, Wasserstand und Gewitterwahrscheinlichkeit.

Speicherung von Benutzereinstellungen

Die Anwendung soll ausgewählte Einstellungen dauerhaft speichern, sodass diese auch nach einem Neustart der App weiterhin verfügbar sind.

Kommunikation mit der bestehenden Backend-Infrastruktur

Die App muss die von der Wetterstation und dem bestehenden Backend bereitgestellten Daten über definierte REST-Schnittstellen abrufen und verarbeiten können.

1.2.2. Nicht-funktionale Anforderungen

Nicht-funktionale Anforderungen beschreiben Qualitätsmerkmale eines Softwaresystems und legen fest, wie bestimmte Funktionen umgesetzt werden sollen.

Benutzerfreundlichkeit

Die Anwendung soll eine intuitive und leicht verständliche Benutzeroberfläche bieten. Die Navigation zwischen den verschiedenen Ansichten soll ohne Vorkenntnisse möglich sein.

Performance

Aktuelle Wetterdaten und Vorhersagen sollen innerhalb kurzer Zeit geladen und dargestellt werden. Lange Ladezeiten oder spürbare Verzögerungen sollen vermieden werden.

Plattformunabhängigkeit

Die Anwendung soll sowohl auf Android- als auch auf iOS-Geräten lauffähig sein. Um dieses Ziel zu erreichen, wurde das plattformübergreifende Framework Flutter eingesetzt.

Wartbarkeit

Der Quellcode soll modular aufgebaut sein und klar definierte Verantwortlichkeiten besitzen. Dadurch sollen zukünftige Erweiterungen und Anpassungen erleichtert werden.

Zuverlässigkeit

Fehlerhafte oder nicht verfügbare Datenquellen sollen durch geeignete Fehlermeldungen behandelt werden, ohne dass die Anwendung abstürzt oder ihre Funktion vollständig verliert.

1.2.3. Zielgruppe

Die Zielgruppe der Anwendung umfasst Personen, die regelmäßig Informationen über lokale Wetterbedingungen und Wasserstände benötigen. Hierzu zählen insbesondere Wassersportler:innen, Angler:innen, Freizeitnutzende sowie interessierte Anwohnerinnen und Anwohner der Region. Darüber hinaus richtet sich die Anwendung an user mit erhöhtem Informationsbedarf, die im Expertenmodus detaillierte meteorologische Analysen durchführen möchten.

1.3. Systemarchitektur

Die Architektur einer Software beschreibt die grundlegende Struktur des Systems sowie die Beziehungen zwischen den einzelnen Komponenten. Ziel einer geeigneten Softwarearchitektur ist es, die Komplexität des Systems zu reduzieren, Verantwortlichkeiten klar zu trennen und die Wartbarkeit sowie Erweiterbarkeit der Anwendung zu verbessern.

Für die Entwicklung der FogCast-App wurde eine mehrschichtige Architektur gewählt, die sich an den Prinzipien der Separation of Concerns orientiert. Dabei werden Benutzeroberfläche, Datenverarbeitung und Datenzugriff voneinander getrennt. Diese Struktur erleichtert sowohl die Weiterentwicklung der Anwendung als auch die Fehlersuche und Wartung.

Architekturübersicht

Die FogCast-App kommuniziert mit einem bereits vorhandenen Backend, das Wetterdaten der Wetterstation sowie Wettervorhersagen bereitstellt. Die mobile Anwendung fungiert als Client und ruft die benötigten Informationen über REST-Schnittstellen ab.

Der grundlegende Datenfluss lässt sich wie folgt darstellen:

Wetterstation
↓
Backend / REST API
↓
API-Schicht
↓
Repository-Schicht
↓
Riverpod State Management
↓
Benutzeroberfläche (Flutter UI)

Die einzelnen Ebenen übernehmen dabei klar definierte Aufgaben und sind möglichst unabhängig voneinander implementiert.

Benutzeroberfläche

Die Präsentationsschicht wurde mit dem Framework Flutter umgesetzt. Flutter ermöglicht die Entwicklung plattformübergreifender Anwendungen auf Basis einer gemeinsamen Codebasis für Android und iOS. Die Benutzeroberfläche wird vollständig durch Widgets beschrieben, die hierarchisch aufgebaut sind und den aktuellen Zustand der Anwendung visualisieren.

Zu den zentralen Ansichten der Anwendung gehören:

- Startseite mit aktuellen Wetterdaten
- Mehrtägige Wettervorhersage
- Detailansicht einzelner Vorhersagetage
- Expertenmodus mit erweiterten Analysefunktionen
- Einstellungs- und Konfigurationsmenü

Die Benutzeroberfläche greift nicht direkt auf externe Datenquellen zu, sondern bezieht sämtliche Informationen über das State Management.

State Management mit Riverpod

Zur Verwaltung des Anwendungszustands wurde das Riverpod-Framework eingesetzt. Riverpod stellt Daten zentral bereit und ermöglicht eine automatische Aktualisierung der Benutzeroberfläche bei Änderungen der zugrunde liegenden Daten.

Der Einsatz von Riverpod bietet mehrere Vorteile:

- Trennung von Geschäftslogik und Benutzeroberfläche
- Vermeidung unnötiger Datenabfragen
- Verbesserte Testbarkeit einzelner Komponenten
- Automatische Aktualisierung abhängiger Widgets

Die verschiedenen Ansichten der Anwendung greifen über Provider auf die benötigten Wetter- und Vorhersagedaten zu. Änderungen des Zustands werden automatisch an die entsprechenden Widgets weitergegeben.

Repository-Schicht

Zwischen der Benutzeroberfläche und der eigentlichen Datenquelle befindet sich eine Repository-Schicht. Diese dient als Vermittler zwischen den einzelnen Systemebenen und kapselt den Zugriff auf externe Datenquellen.

Die Repository-Klassen übernehmen insbesondere folgende Aufgaben:

- Abruf von Daten über die API-Schicht
- Verarbeitung von Fehlerfällen
- Umwandlung der empfangenen Daten in interne Datenmodelle
- Bereitstellung einheitlicher Schnittstellen für die Anwendung

Durch diese zusätzliche Abstraktionsschicht wird verhindert, dass Komponenten der Benutzeroberfläche direkt von den technischen Details der API abhängig sind.

API-Schicht

Die Kommunikation mit dem Backend erfolgt über REST-basierte HTTP-Anfragen. Hierfür werden spezialisierte API-Klassen eingesetzt, die für den Abruf bestimmter Datentypen verantwortlich sind.

Zu den wichtigsten Schnittstellen gehören:

- Abruf aktueller Wetterdaten
- Abruf von Wettervorhersagen
- Abruf historischer Wasserstandsdaten

Die API-Schicht übernimmt die Erstellung der HTTP-Anfragen, die Verarbeitung der Serverantworten sowie die Umwandlung der empfangenen JSON-Daten in Dart-Objekte.

Datenmodelle

Für die interne Verarbeitung der Wetterdaten werden Data Transfer Objects (DTOs) verwendet. Diese stellen strukturierte Datenmodelle dar, die die vom Backend gelieferten Informationen in einer für die Anwendung nutzbaren Form repräsentieren.

Beispiele hierfür sind:

- ForecastDto
- LiveDataDto
- HistoryDto

Die Verwendung von DTOs verbessert die Typensicherheit der Anwendung und reduziert Fehler bei der Verarbeitung externer Daten.

Persistente Speicherung von Einstellungen

Benutzerspezifische Einstellungen werden lokal auf dem Endgerät gespeichert. Hierfür wird das Flutter-Paket SharedPreferences verwendet.

Gespeichert werden unter anderem:

- ausgewählte Diagramme im Expertenmodus
- gewähltes Wettermodell
- bevorzugte Seitenansicht

Dadurch bleiben individuelle Konfigurationen auch nach dem Schließen der Anwendung erhalten und müssen nicht erneut vorgenommen werden.

Architekturvorteile

Die gewählte Architektur bietet mehrere Vorteile für die Weiterentwicklung der Anwendung. Durch die klare Trennung der Verantwortlichkeiten können einzelne Komponenten unabhängig voneinander angepasst oder erweitert werden. Neue Datenquellen, zusätzliche Diagramme oder weitere Wetterparameter lassen sich ohne grundlegende Änderungen an der Benutzeroberfläche integrieren.

Darüber hinaus verbessert die Struktur die Wartbarkeit des Projekts und erleichtert zukünftigen Entwicklerinnen und Entwicklern die Einarbeitung in den bestehenden Quellcode.

1.4. Technologie-Stack

Für die Entwicklung der FogCast-App wurden verschiedene Technologien und Bibliotheken eingesetzt, die eine plattformübergreifende Entwicklung sowie eine effiziente Datenverarbeitung ermöglichen.

Flutter und Dart

Die Anwendung wurde mit dem Framework Flutter und der Programmiersprache Dart entwickelt. Flutter ermöglicht die Entwicklung nativer Android- und iOS-Anwendungen auf Basis einer gemeinsamen Codebasis und reduziert dadurch den Entwicklungsaufwand erheblich.

Riverpod

Für das State Management wurde Riverpod verwendet. Riverpod sorgt für eine zentrale Verwaltung des Anwendungszustands und ermöglicht die automatische Aktualisierung der Benutzeroberfläche bei Änderungen der Daten.

Datenvisualisierung

Zur grafischen Darstellung meteorologischer Daten wird die Bibliothek fl_chart eingesetzt. Sie ermöglicht die Erstellung von Linien- und Balkendiagrammen für Temperatur, Luftfeuchtigkeit, Niederschlag, Wolkendichte und Wasserstand.

Lokale Datenspeicherung

Benutzereinstellungen werden mithilfe von SharedPreferences lokal auf dem Endgerät gespeichert. Dadurch bleiben individuelle Konfigurationen auch nach dem Neustart der Anwendung erhalten.

1.5. Fazit und Ausblick

Im Rahmen des Projekts wurde die mobile Anwendung FogCast entwickelt, die auf einer bereits bestehenden Wetterstation und der dazugehörigen Webanwendung aufbaut. Ziel war es, die vorhandenen Wetter- und Prognosedaten für mobile Endgeräte zugänglich zu machen und eine benutzerfreundliche Alternative zur bestehenden Webanwendung bereitzustellen.

Die entwickelte App ermöglicht die Darstellung aktueller Wetterdaten, mehrtägiger Wettervorhersagen sowie detaillierter Tagesansichten. Darüber hinaus wurde ein Expertenmodus implementiert, der erweiterte meteorologische Analysen und die Visualisierung verschiedener Wetterparameter ermöglicht. Durch die Verwendung von Flutter konnte eine plattformübergreifende Lösung geschaffen werden, die sowohl auf Android- als auch auf iOS-Geräten eingesetzt werden kann.

Besonders hervorzuheben ist die modulare Architektur der Anwendung. Die Trennung von Benutzeroberfläche, Datenverwaltung und Datenzugriff erleichtert die Wartung und schafft eine solide Grundlage für zukünftige Erweiterungen. Gleichzeitig konnten moderne Technologien wie Riverpod, REST-Schnittstellen und interaktive Diagrammvisualisierungen erfolgreich integriert werden.

Trotz des erreichten Funktionsumfangs bestehen verschiedene Möglichkeiten zur Weiterentwicklung der Anwendung. So könnten künftig zusätzliche Wetterparameter integriert

oder bestehende Visualisierungen erweitert werden. Ebenso wäre die Unterstützung mehrerer Wetterstationen denkbar, um Wetterdaten aus verschiedenen Standorten innerhalb einer Anwendung darzustellen.

Ein weiterer möglicher Entwicklungsschritt besteht in der Einführung von Benachrichtigungsfunktionen. Nutzer könnten beispielsweise automatisch über Starkregenereignisse, kritische Wasserstände oder andere relevante Wetterentwicklungen informiert werden. Darüber hinaus könnte eine Offline-Funktionalität implementiert werden, um zuletzt geladene Daten auch ohne aktive Internetverbindung verfügbar zu machen.

Zusammenfassend zeigt das Projekt, wie eine bestehende technische Infrastruktur erfolgreich um eine moderne mobile Anwendung erweitert werden kann. Die entwickelte FogCast-App erfüllt die definierten Anforderungen und bietet eine benutzerfreundliche Möglichkeit zur Darstellung und Analyse meteorologischer Daten. Gleichzeitig bildet sie eine geeignete Grundlage für zukünftige Erweiterungen und die Weiterentwicklung durch nachfolgende Projektgruppen.

2. Technische Dokumentation

2.1. Softwarearchitektur

Die FogCast-App wurde nach einer mehrschichtigen Softwarearchitektur entwickelt. Ziel dieses Aufbaus ist es, die einzelnen Aufgabenbereiche der Anwendung voneinander zu trennen und dadurch eine übersichtliche, wartbare und leicht erweiterbare Codebasis zu schaffen. Jede Schicht übernimmt dabei eine klar definierte Verantwortung und kommuniziert ausschließlich mit den benachbarten Schichten.

Die Benutzeroberfläche dient ausschließlich der Darstellung der Wetterinformationen und der Interaktion mit den Nutzerinnen und Nutzern. Die eigentliche Verarbeitung der Daten sowie die Kommunikation mit dem Backend erfolgen in separaten Komponenten. Dadurch bleibt die Oberfläche unabhängig von der Datenbeschaffung und kann bei Bedarf angepasst werden, ohne Änderungen an der Backend-Anbindung vornehmen zu müssen.

Der grundlegende Datenfluss innerhalb der Anwendung verläuft nach folgendem Schema:

Benutzeroberfläche → State Management → Repository → API → Backend

Nachdem die Daten vom Backend empfangen wurden, werden sie in umgekehrter Richtung wieder an die Benutzeroberfläche zurückgegeben und dort dargestellt. Diese Architektur ermöglicht eine klare Trennung der Verantwortlichkeiten und erleichtert sowohl die Wartung als auch zukünftige Erweiterungen der Anwendung.

3.2 Einstiegspunkt der Anwendung

Der Einstiegspunkt der Anwendung befindet sich in der Datei `main.dart`. Nach dem Start der App wird zunächst ein `ProviderScope` erzeugt, der sämtliche Riverpod-Provider für die gesamte Anwendung verfügbar macht. Anschließend wird die Hauptklasse `FogCastApp` geladen, welche die MaterialApp initialisiert und die Navigation innerhalb der Anwendung vorbereitet.

Von dort aus wird die Startseite der Anwendung geöffnet. Alle weiteren Seiten greifen auf denselben zentralen Zustand zu und können dadurch auf bereits geladene Wetterdaten zugreifen, ohne diese erneut vom Backend abrufen zu müssen.

3.3 State Management

Für die Verwaltung des Anwendungszustands wird das Riverpod-Framework verwendet. Dadurch stehen die benötigten Daten allen Bereichen der Anwendung zentral zur Verfügung und Änderungen werden automatisch an die Benutzeroberfläche weitergegeben.

Die Klasse `LiveDataState` speichert den aktuellen Zustand der Anwendung. Hierzu gehören unter anderem der Ladezustand, mögliche Fehlermeldungen sowie die aktuell verfügbaren Wetterdaten und Vorhersagen. Die Benutzeroberfläche beobachtet diesen Zustand kontinuierlich und aktualisiert sich automatisch, sobald neue Daten geladen oder bestehende Daten verändert werden.

Die Geschäftslogik befindet sich in der Klasse `LiveDataNotifier`. Diese ist für das Abrufen der Wetterdaten verantwortlich und übernimmt die Aktualisierung des Zustands. Beim Start der Anwendung werden sowohl aktuelle Messwerte als auch Wettervorhersagen geladen und anschließend im zentralen State gespeichert.

Durch diese Trennung zwischen Datenverwaltung und Benutzeroberfläche bleibt der Code übersichtlich und Änderungen an der Datenverarbeitung wirken sich nicht unmittelbar auf die Darstellung der Informationen aus.

3.4 API-Schicht

Die Kommunikation mit dem Backend erfolgt ausschließlich über spezialisierte API-Klassen. Jede dieser Klassen ist für einen bestimmten Datentyp verantwortlich und kapselt sämtliche HTTP-Anfragen.

Die Klasse `LiveDataApi` lädt aktuelle Messwerte der Wetterstation sowie weitere Stationsinformationen. `ForecastApi` ruft Wettervorhersagen verschiedener Wettermodelle ab, während `HistoryApi` historische Messwerte, beispielsweise für den Wasserstand, bereitstellt.

Die API-Klassen übernehmen ausschließlich den Datenaustausch mit dem Backend. Sie erstellen die entsprechenden HTTP-Anfragen, empfangen die Antworten des Servers und stellen diese für die weitere Verarbeitung zur Verfügung. Dadurch bleibt die Kommunikation mit dem Backend zentral organisiert und Änderungen an den Schnittstellen müssen lediglich innerhalb der jeweiligen API-Klasse vorgenommen werden.

3.5 Repository-Schicht

Zwischen der API-Schicht und dem State Management befindet sich die Repository-Schicht. Sie dient als Vermittler zwischen dem Backend und der restlichen Anwendung.

Für jede Datenquelle existiert ein eigenes Repository. Das `LiveDataRepository` verarbeitet aktuelle Wetterdaten, das `ForecastRepository` stellt Wettervorhersagen bereit und das `HistoryRepository` übernimmt den Zugriff auf historische Messwerte.

Die Repositories rufen die entsprechenden API-Klassen auf, verarbeiten die empfangenen Daten und geben sie in einer Form zurück, die von der Anwendung direkt genutzt werden kann. Dadurch muss weder das State Management noch die Benutzeroberfläche wissen, wie die Kommunikation mit dem Backend im Detail funktioniert.

3.6 Datenmodelle (DTOs)

Die vom Backend gelieferten JSON-Daten werden innerhalb der Anwendung in sogenannten Data Transfer Objects (DTOs) gespeichert. Diese Klassen stellen eine typisierte Repräsentation der empfangenen Daten dar und erleichtern deren Weiterverarbeitung.

Zu den wichtigsten Datenmodellen gehören `LiveDataDto`, `ForecastDto`, `HistoryDto` sowie `WeatherStationDto`. Sie enthalten sämtliche meteorologischen Messwerte und Vorhersageinformationen, die innerhalb der App dargestellt werden.

Durch die Verwendung von DTOs wird sichergestellt, dass Daten strukturiert verarbeitet werden können und keine direkte Arbeit mit unübersichtlichen JSON-Objekten erforderlich ist.

3.7 Benutzeroberfläche

Die Benutzeroberfläche besteht aus mehreren eigenständigen Seiten, die jeweils eine klar definierte Aufgabe erfüllen.

Die Startseite zeigt die wichtigsten aktuellen Wetterinformationen sowie eine kompakte Vorhersage für die kommenden Tage. Über die Detailansicht können Nutzerinnen und Nutzer stündliche Wettervorhersagen eines ausgewählten Tages betrachten.

Der Expertenmodus stellt zusätzlich umfangreiche Diagramme für verschiedene meteorologische Parameter bereit. Hierzu gehören Temperatur, Luftfeuchtigkeit, Niederschlag, Wolkendichte, Gewitterwahrscheinlichkeit sowie historische Wasserstandsdaten. Darüber hinaus können Anwender auswählen, welche Diagramme angezeigt werden sollen und welches Wettermodell verwendet werden soll.

Alle Seiten greifen auf denselben zentralen Anwendungszustand zu, wodurch doppelte Datenabfragen vermieden werden und Änderungen unmittelbar in der gesamten Anwendung sichtbar werden.

3.8 Persistente Speicherung

Bestimmte Benutzereinstellungen werden dauerhaft auf dem Endgerät gespeichert. Hierfür wird die Flutter-Bibliothek `SharedPreferences` verwendet.

Gespeichert werden unter anderem die Auswahl der im Expertenmodus angezeigten Diagramme sowie weitere benutzerspezifische Einstellungen. Beim erneuten Start der Anwendung werden diese Einstellungen automatisch geladen und wiederhergestellt, sodass keine erneute Konfiguration erforderlich ist.

3.9 Erweiterbarkeit

Der modulare Aufbau der Anwendung erleichtert zukünftige Erweiterungen erheblich. Neue Wetterparameter oder zusätzliche Diagramme können durch das Hinzufügen neuer API-Methoden, Repository-Funktionen und Benutzeroberflächenkomponenten integriert werden, ohne bestehende Funktionalitäten wesentlich zu verändern.

Ebenso ermöglicht die Architektur die Anbindung weiterer Wetterstationen oder zusätzlicher Datenquellen. Durch die klare Trennung zwischen Benutzeroberfläche, Datenverwaltung und Datenzugriff können einzelne Komponenten unabhängig voneinander weiterentwickelt und getestet werden. Dies bildet eine solide Grundlage für die langfristige Wartung und den Ausbau der FogCast-App.